

REGRESSION ANALYSIS REPORT

Simple Linear • Multiple Linear • Gradient Boosting

Machine Learning Practice Project – Supervised Learning

Python | scikit-learn | pandas | matplotlib

November 2026

Table of Contents

Table of Contents 2

1. Executive Summary 3

2. Project Overview & Methodology 4

3. Evaluation Metrics Explained 5

4. Simple Linear Regression 6

5. Multiple Linear Regression 8

6. Gradient Boosting Regressor 10

7. Comparative Analysis 12

8. Regression Model Code 14

9. Conclusions & Recommendations 16

1. Executive Summary

This report documents a structured, hands-on study of three supervised regression techniques implemented in Python: **Simple Linear Regression**, **Multiple Linear Regression** and the **Gradient Boosting Regressor**. Each technique was applied to a distinct, realistic prediction problem – forecasting exam scores from study time, estimating used-car prices from vehicle attributes, and predicting laptop prices from hardware specifications.

Across all three experiments the workflow followed industry best practice: data loading and validation, exploratory visualisation, a train/test split, model fitting, prediction, and quantitative evaluation using R^2, MAE, MSE and RMSE. Every model generalised well to unseen data, and the non-linear ensemble model delivered the strongest result.

0.9526	0.9199	0.9842
Simple Linear	Multiple Linear	Gradient Boosting
Test R^2	Test R^2	Test R^2

Key Findings

- **Study time is a strong predictor of exam score:** each additional study hour adds about 6.81 marks (intercept 15.18), with a test R^2 of 0.9526.
- **Car prices are driven by engine size and age:** every extra litre of engine adds roughly $3,240 while each additional year of age reduces price by about $845.
- **RAM dominates laptop pricing:** RAM accounts for 68.7% of the Gradient Boosting model's feature importance, followed by storage at 27.6%.
- **Ensemble learning wins on complex data:** Gradient Boosting reached $R^2 = 0.9842$ with an average error of only $48.85.

Bottom line: all three models are reliable on their respective datasets. Linear models offer transparency and interpretability; Gradient Boosting offers the highest predictive accuracy.

2. Project Overview & Methodology

Regression is the branch of supervised machine learning concerned with predicting a continuous numeric target from one or more input features. The objective of this project was to build practical competence in the complete regression workflow and to compare simple, interpretable models with a more flexible ensemble method.

Datasets Used

Section	Dataset	Shape	Inputs	Target
1 – Simple Linear	study_hours_exam_score.csv	1,000 × 2	StudyHours	ExamScore
2 – Multiple Linear	car_price_multiple.csv	5,000 × 4	EngineSize_L, Mileage_km, CarAge_years	Price_USD
3 – Gradient Boosting	laptop_price_gbr.csv	10,000 × 6	RAM_GB, Storage_GB, ScreenSize_inch, Weight_kg, BatteryLife_hrs	Price_USD

Workflow



Figure 1 – End-to-end regression workflow applied to every dataset

Tools & Libraries

- **pandas / NumPy** – data loading, validation and numerical operations
- **matplotlib** – scatter plots, regression lines and diagnostic charts
- **scikit-learn** – `LinearRegression`, `GradientBoostingRegressor`, `train_test_split` and the metrics module

3. Evaluation Metrics Explained

Choosing the right metric is as important as choosing the right model. The following measures were used throughout the project.

Metric	Definition	Interpretation
R^2 Score	Proportion of variance in the target explained by the model	Closer to 1.0 is better; 0 equals predicting the mean
MAE	Mean of absolute differences between actual and predicted values	Average error in the original units; robust to outliers
MSE	Mean of squared errors	Penalises large errors heavily; units are squared
RMSE	Square root of MSE	Error in original units; sensitive to large mistakes

Train vs. Test Performance

Every model was fitted on a training partition and scored on a held-out test partition. Comparing the two scores is the primary check against overfitting: a large drop from training to testing would signal that the model memorised noise rather than learning a general pattern. In this project the training and test scores are very close for both linear models, confirming healthy generalisation.

Data Quality Checks

Before modelling, each dataset was inspected with `.shape`, `.info()`, `.describe()` and `.isnull().sum()`. All three datasets – 1,000, 5,000 and 10,000 records respectively – contained **zero missing values**, so no imputation was required.

4. Simple Linear Regression

Problem: predict a student's *ExamScore* from the number of *StudyHours*. Simple Linear Regression fits a straight line `y = m·x + c` by minimising the sum of squared residuals (Ordinary Least Squares).

Exploratory Analysis

A scatter plot of StudyHours against ExamScore shows a clear, strongly positive and approximately linear relationship, making this an ideal candidate for a linear model.

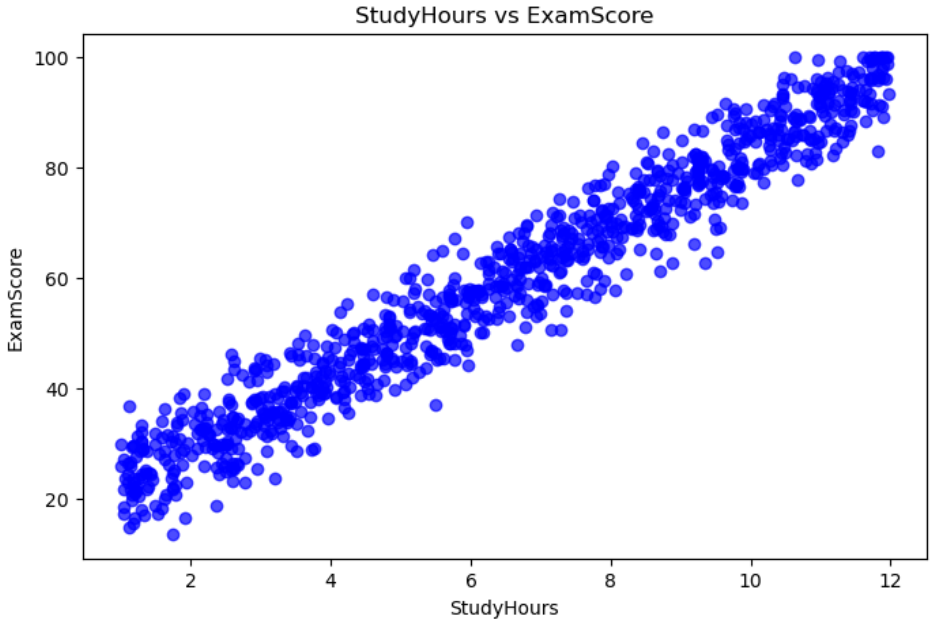


Figure 2 – StudyHours vs. ExamScore scatter plot (from the notebook)

Model Parameters

Parameter	Value	Meaning
Slope (m)	6.8079	Each extra study hour raises the expected score by ≈ 6.81 marks
Intercept (c)	15.1760	Expected score with zero study hours
Training R^2	0.9462	94.6% of variance explained on training data
Testing R^2	0.9526	95.3% of variance explained on unseen data

Fitted Line & Predictions

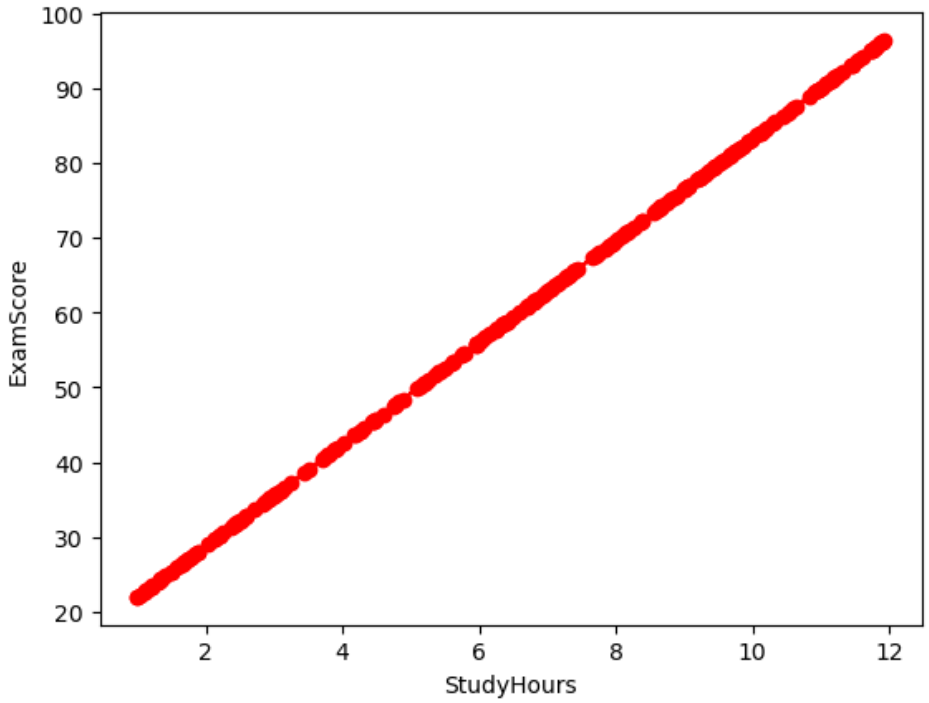


Figure 3 – Predicted test-set values and the best-fit regression line (from the notebook)

Interpretation

The test R^2 (0.9526) is marginally higher than the training R^2 (0.9462), which is entirely normal for a train/test split on a small, clean dataset and indicates there is no overfitting. The residuals, computed as `y_test − predicted`, are centred near zero across the 250 test observations, showing the model is unbiased. Occasional larger errors (for example, around –11.5 marks) reflect natural individual variation in student performance that study time alone cannot explain.

> **Takeaway:** a single feature explains over 95% of the variation in exam scores – a simple, transparent and highly effective model for this dataset.

5. Multiple Linear Regression

Problem: estimate a used car's *Price_USD* from its engine size, mileage and age. Multiple Linear Regression generalises the simple model to several predictors: `y = b0 + b1·x1 + b2·x2 + b3·x3`.

Dataset Profile

Statistic	EngineSize_L	Mileage_km	CarAge_years	Price_USD
Mean	2.48	99,139.7	7.54	22,119.5
Std. dev.	0.86	58,459.5	4.33	5,432.7
Minimum	1.00	514	0.0	5,820
Median	2.45	98,220.5	7.6	22,155.4

The dataset contains 5,000 records split 80/20, giving 4,000 training and 1,000 testing observations. No missing values were present.

Learned Coefficients

Feature	Coefficient	Business interpretation
EngineSize_L	+3,240.16	Each additional litre increases price by ≈ $3,240
CarAge_years	–845.09	Each additional year of age lowers price by ≈ $845
Mileage_km	–0.0452	Each additional km lowers price by ≈ 4.5 cents
Intercept	24,936.44	Baseline price when all features are zero

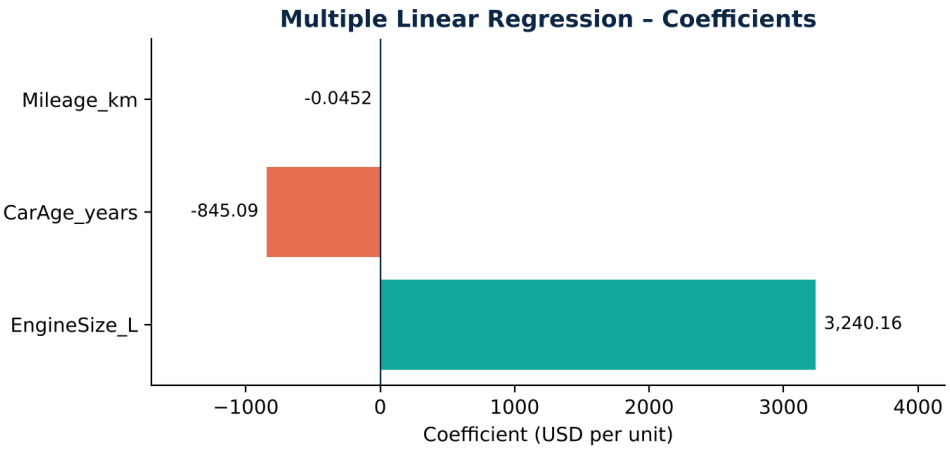


Figure 4 – Regression coefficients (raw scale)

Which Feature Matters Most?

By raw coefficient magnitude, **EngineSize_L** has the strongest influence. A caution applies: the coefficients are on the original, unscaled units, so Mileage_km appears tiny only because its values run into the hundreds of thousands. A fair comparison of feature importance would use standardised features; however, engine size and vehicle age are clearly the dominant, intuitive price drivers here.

Performance Metrics

Metric	Value
Training R^2	0.9241
Testing R^2	0.9199
MAE	$1,221.32
MSE	2,358,416.64
RMSE	$1,535.71

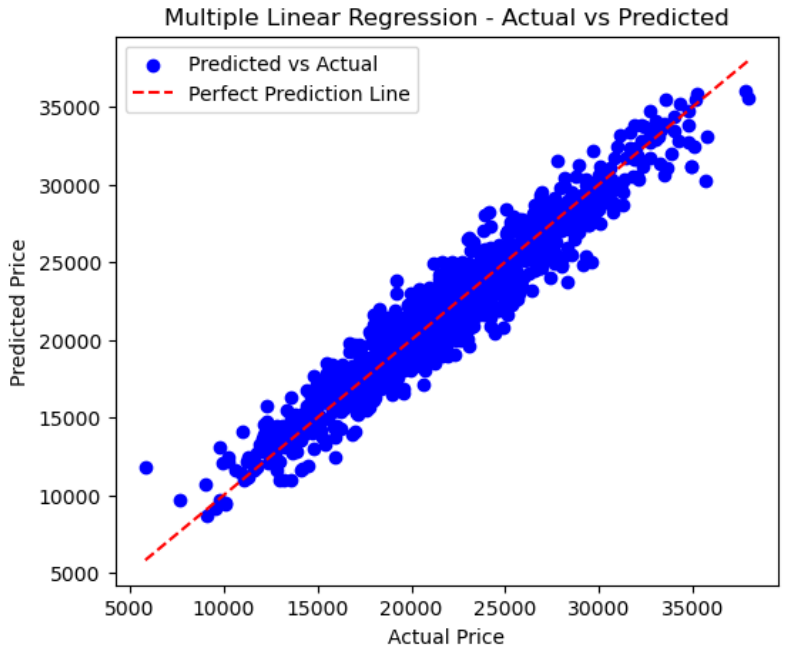


Figure 5 – Actual vs. predicted prices with the perfect-prediction diagonal (from the notebook)

Interpretation

The model explains about 92% of price variance, and on average its predictions miss the true price by roughly $1,221 on cars that cost about $22,000 – an error near 5.5%. The small gap between training (0.9241) and testing (0.9199) R^2 confirms stable generalisation. Points that hug the red diagonal in Figure 5 indicate accurate predictions.

6. Gradient Boosting Regressor

Problem: predict a laptop's *Price_USD* from RAM, storage, screen size, weight and battery life. Gradient Boosting builds an ensemble of shallow decision trees sequentially; each new tree is trained to correct the residual errors of the ensemble built so far, which lets the model capture non-linear effects and feature interactions.

Model Configuration

Hyper-parameter	Value	Role
n_estimators	100	Number of boosting stages (trees)
learning_rate	0.1	How much each tree corrects the previous ensemble
max_depth	3	Complexity of each individual tree
random_state	42	Ensures reproducible results
Train/Test split	80 / 20 (8,000 / 2,000)	Held-out evaluation set

Performance

0.9842	$48.85	$61.03
R^2 score	MAE	RMSE

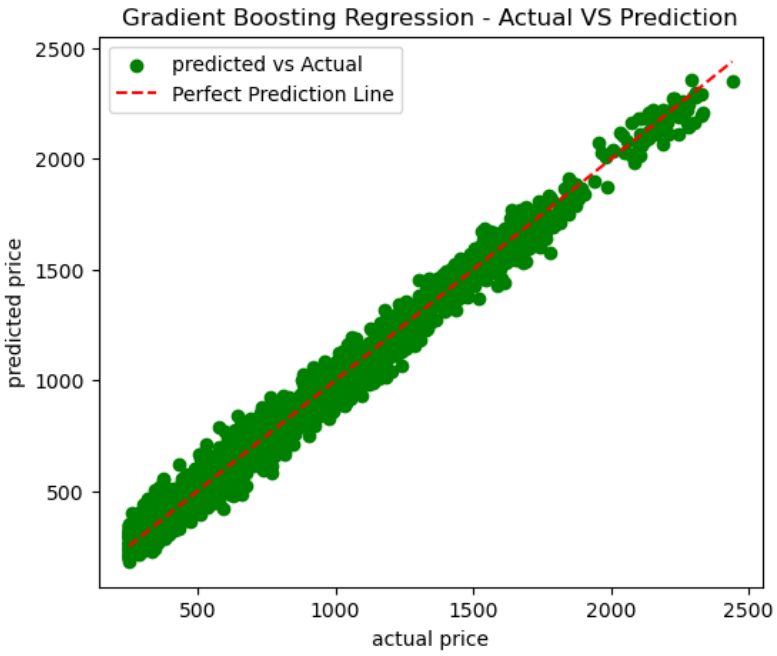


Figure 6 – Gradient Boosting actual vs. predicted prices (from the notebook)

Feature Importance

Tree ensembles provide a built-in measure of how much each feature contributes to reducing prediction error. **RAM_GB** is by far the most influential feature (68.7%), followed by **Storage_GB** (27.6%). Together they account for over 96% of the model's predictive power, while weight, battery life and screen size contribute only marginally.

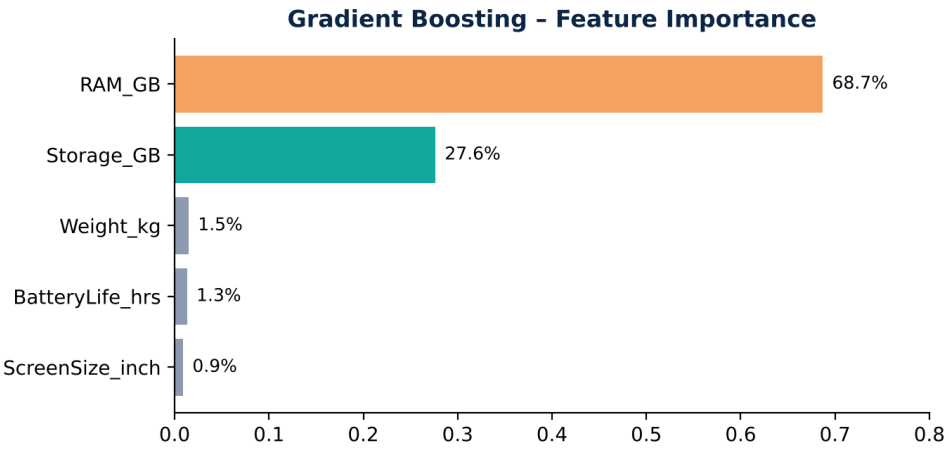


Figure 7 – Feature importances from the trained Gradient Boosting model

Rank	Feature	Importance
1	RAM_GB	0.6869
2	Storage_GB	0.2763
3	Weight_kg	0.0148
4	BatteryLife_hrs	0.0131
5	ScreenSize_inch	0.0089

Insight: for pricing or product-planning decisions, memory and storage configuration are the levers that move laptop prices most.

7. Comparative Analysis

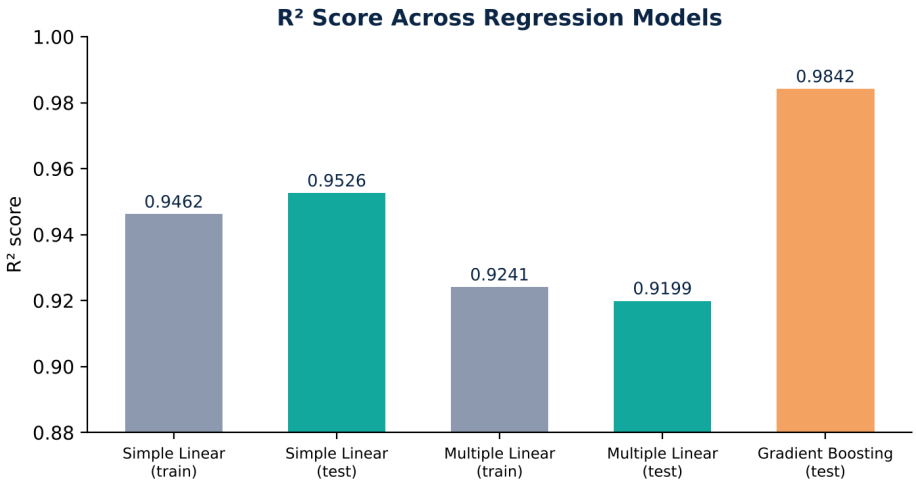


Figure 8 – R^2 comparison across the three regression models

Aspect	Simple Linear	Multiple Linear	Gradient Boosting
Features	1	3	5
Observations	1,000	5,000	10,000
Test R^2	0.9526	0.9199	0.9842
Interpretability	Very high	High	Moderate
Handles non-linearity	No	No	Yes
Training cost	Minimal	Minimal	Higher
Best used when	One dominant linear driver	Several additive drivers	Complex, interacting features

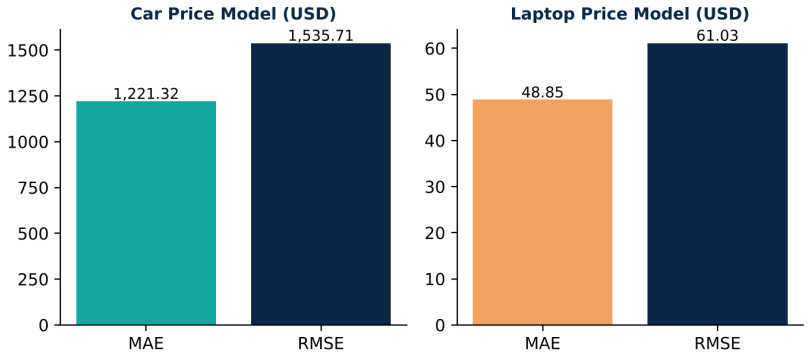


Figure 9 – Absolute error metrics (MAE & RMSE) for the car and laptop price models

Important caveat: the three models were trained on *different datasets*, so their R^2 values are not a like-for-like benchmark. The comparison illustrates each method's behaviour; a fair head-to-head requires fitting all models to the same data.

Linear vs. Ensemble Models (Conceptual)

Multiple Linear Regression assumes a linear, additive relationship between features and target. It is fast, transparent and easy to explain, but cannot represent curved relationships or feature interactions without manual feature engineering.

Gradient Boosting combines many shallow trees, each fixing the errors of its predecessors. It automatically captures non-linearities and interactions – such as the combined effect of RAM *and* storage on price – and therefore tends to achieve higher R^2 whenever the underlying relationship is non-linear. The price paid is reduced interpretability and a greater need for hyper-parameter tuning.

Strengths & Limitations

Model	Strengths	Limitations
Simple Linear	Easiest to explain; closed-form solution; fast	Only one predictor; assumes linearity
Multiple Linear	Interpretable coefficients; low variance	Assumes additivity; sensitive to multicollinearity and outliers
Gradient Boosting	High accuracy; captures interactions; feature importance	Risk of overfitting; slower; harder to interpret

8. Regression Model Code

The following listings show the core implementation of each of the three regression models, exactly as used in the notebook.

8.1 Simple Linear Regression

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df = pd.read_csv("study_hours_exam_score.csv")
X = df[["StudyHours"]]          # independent variable
y = df[["ExamScore"]]           # dependent variable

X_train, X_test, y_train, y_test = train_test_split(X, y)

lin = LinearRegression()
lin.fit(X_train, y_train)

print(lin.coef_)                # slope (m)      -> 6.8079
print(lin.intercept_)           # intercept (c)  -> 15.1760
predicted = lin.predict(X_test)
print(lin.score(X_test, y_test))   # test R2     -> 0.9526
```

8.2 Multiple Linear Regression

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

df1 = pd.read_csv("car_price_multiple.csv")
X = df1[["EngineSize_L", "Mileage_km", "CarAge_years"]]
y = df1[["Price_USD"]]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

multi = LinearRegression()
multi.fit(X_train, y_train)
pred = multi.predict(X_test)

mse  = mean_squared_error(y_test, pred)
mae  = mean_absolute_error(y_test, pred)
rmse = np.sqrt(mse)
r2   = r2_score(y_test, pred)      # -> 0.9199
```

8.3 Gradient Boosting Regressor

```
from sklearn.ensemble import GradientBoostingRegressor

df2 = pd.read_csv("laptop_price_gbr.csv")
X = df2[["RAM_GB", "Storage_GB", "ScreenSize_inch",
         "Weight_kg", "BatteryLife_hrs"]]
y = df2[["Price_USD"]]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

gbr = GradientBoostingRegressor(
    n_estimators=100,     # number of trees
    learning_rate=0.1,    # contribution of each tree
    max_depth=3,          # complexity of each tree
    random_state=42)
gbr.fit(X_train, y_train)

y_pred = gbr.predict(X_test)
print("R2  :", r2_score(y_test, y_pred))   # 0.9842
print("MAE :", mean_absolute_error(y_test, y_pred))  # 48.85
```

9. Conclusions & Recommendations

This project successfully demonstrated the full regression lifecycle on three datasets of increasing size and complexity. All models achieved R^2 scores above 0.91 on unseen data with minimal gap between training and testing performance, indicating well-fitted, generalisable models.

Conclusions

- Simple Linear Regression explained 95.3% of exam-score variance using study hours alone.
- Multiple Linear Regression produced interpretable price drivers (engine size up, age down) and 92.0% explained variance.
- Gradient Boosting delivered the best accuracy ($R^2 = 0.9842$, MAE ≈ $49) and exposed RAM and storage as the principal pricing factors.

Recommendations for Future Work

- **Cross-validation:** use k-fold validation for more robust performance estimates instead of a single split.
- **Fix random seeds:** set `random_state` in the linear-model splits so results are exactly reproducible.
- **Feature scaling:** standardise inputs to compare coefficient importance fairly.
- **Hyper-parameter tuning:** apply `GridSearchCV` to optimise n_estimators, learning_rate and max_depth.
- **Residual diagnostics:** add residual plots and normality checks to validate linear-regression assumptions.
- **Benchmark on common data:** fit all three models on one dataset for a like-for-like comparison, and try regularised models (Ridge, Lasso) and Random Forests.

Glossary

Term	Meaning
Regression	Predicting a continuous numeric value from input features
Residual	Difference between an actual and a predicted value
Overfitting	A model that fits training noise and performs poorly on new data
Ensemble	A model built by combining many simpler models
Feature importance	A measure of how much each input contributes to predictions